

PREFECT PIPELINES,  
ENGINEERED

# Prefect: Long-running flows that never lose their place.

Do you really need an "orchestration server" to run your pipelines? Rediscover the power...

**+117k**

downloads

**<50  
MB**

RAM, total

**SQLite**

WAL state

THE CONTEXT

# Where it hurts today

**Long runs lose state**

**Workers + server + fleet**

**Observability gets costly**

**THE SHIFT**

# What you gain



## Code-first

Pipelines are Python — versionable, testable.



## Resume, don't restart

Every step checkpoints to the WAL.

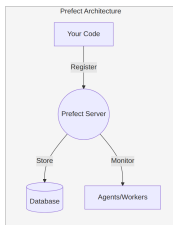
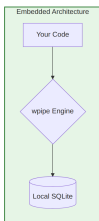


## Featherweight

<50 MB. No broker, no daemon.

## THE ARCHITECTURE

# One pipeline, fully instrumented



State, retries and audit — inside a single run.

## SIDE BY SIDE

## wpipe vs. the status quo

| Category       | Prefect<br>(Server/Cloud)       | wpipe<br>(Embedded<br>Library)      |
|----------------|---------------------------------|-------------------------------------|
| Infrastructure | Requires<br>Server/Agents       | Zero-Config<br>(Self-<br>contained) |
| Persistence    | Postgres / Cloud<br>DB          | SQLite WAL<br>(Local-First)         |
| Dependencies   | External API /<br>Local service | NONE<br>(Standard<br>Python Lib)    |
| Latency        | Network-<br>dependent           | Disk-speed<br>(Atomic<br>writes)    |

SEE IT IN ACTION

## Six lines. That's it.

```
from wpipe import Pipeline, step

@step(name="resume_me",
checkpoint=True)
def resume_me(data):
    return {"state":
hydrate(data["cursor"])}

```

CHECKPOINTS · RETRIES · TIMEOUT ·  
PARALLEL · TRACKER

## Try wpipe in your next pipeline ✓

Install the library, wrap your step  
— checkpoints and docs appear  
by themselves.

```
pip install wpipe
```

---

**wpipe**

Python-native orchestration